

DATA-STRUCTURES & ALGORITHMS

with

Manoj Prabhakaran

Lecture 17

Sorting

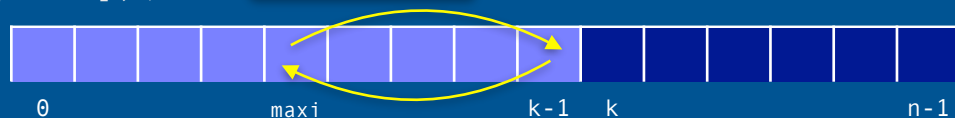
Sorting

- There are several sorting algorithms with asymptotic worst-case costs ranging from $O(n^2)$ to $O(n \log n)$
 - In-place or not
 - In-place: $O(1)$ additional space. Input rearranged into sorted order
 - Stable or not
 - Stable: If there are equal valued keys, they appear in the output in the same order in which they are in the input
 - In practice, concerns beyond worst-case asymptotic behaviour
 - Average case costs. Performance on small inputs.

Selection Sort

- Recall the heap sort approach:
 - At each iteration identify the max, and move it to its final location
- Selection sort is the simplest implementation of this

```
void selection_sort(std::vector<int>& A) {  
    bool sorted = false;    // optional flag to check for early exit  
    for(int k = A.size(); k>1 && !sorted; --k) { // k items remain to be sorted  
        sorted = true;  
        for (int i=1, maxi=0; i<k; ++i) { //scan remaining items for max's index  
            if(A[i] >= A[maxi]) maxi=i; else sorted = false;  
        }  
        if(maxi!=k-1) std::swap(A[k-1],A[maxi]);  
    }  
}
```



- $O(n^2)$ comparisons. Only $O(n)$ assignments (swaps).
- Not a stable sort (equal items can change order when max item swapped out)
- Can exit early if the array becomes sorted sooner

Selection Sort → Heap Sort

- Instead of $O(n)$ time scan to find max, use a *max-heap*
- $O(\log n)$ time to remove the max item; $O(n \log n)$ time overall

```
void heap_sort(std::vector<int>& A) {  
    makeMaxHeap(A); // a function that turns A into a max-heap in place  
    for(int k = A.size(); k>1 ; --k) { // k items remain to be sorted  
        int x = A[k-1]; A[k-1] = A[0];  
        bubbleDown(A,k-1,0,x); // A[0] is bubbled down in heap A[0..k-1] with key x  
    }  
}
```

Annotations:

- specify how many items in A are part of the heap (points to `k-1` in `bubbleDown`)
- location to bubbleDown and its new key value (points to `0, x` in `bubbleDown`)
- Sending `A[k-1]` to the root (and bubbling down) makes it unstable (points to `A[k-1] = A[0]`)

- Exercise: In-place implementations of the functions `makeMaxHeap` and `bubbleDown` as required above
- In-place, and not stable (like selection sort)

Insertion Sort

- A slightly different idea yielding a **stable sort**
 - We will still maintain a suffix in sorted order
 - But the sorted items needn't be in their final locations
 - In each iteration absorb the last unsorted item (instead of max)
 - *Insert it into the sorted part* (sliding items less than it)

```
void insertion_sort(std::vector<int>& A) {  
    for(int k = A.size(); k>0; --k) { // k items remain to be absorbed  
        int i, x = A[k-1];           // will absorb x into the sorted part  
        for (i=k; i<A.size() && A[i] < x; ++i) // scan for x's spot  
            A[i-1]=A[i];               // while sliding items less than x to the left  
        A[i-1]=x; //outside the loop, i-1 is where x settles in.  
    }  
}
```

Important for stability: use `<`, not `<=`



- Stable, in-place sorting

Tree Sort

- Same as insertion sort, but instead of keeping the sorted part in (the suffix of) a vector, keep it in a **balanced BST**

```
void tree_sort(std::vector<int>& A) {  
    balancedBST tree; // not in-place any more  
    for(int k = 0; k < A.size(); ++k) { // k items remain to be absorbed  
        tree.insert(A[k]);  
    }  
    tree.inorder(A); // overwrite the vector with tree's in-order sequence  
}
```

For stability, requires that a duplicate key inserted later goes to the right of the existing key (and stays so while balancing)

- $O(n \log n)$ time, stable
- Needs $O(n)$ additional space: not in-place

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

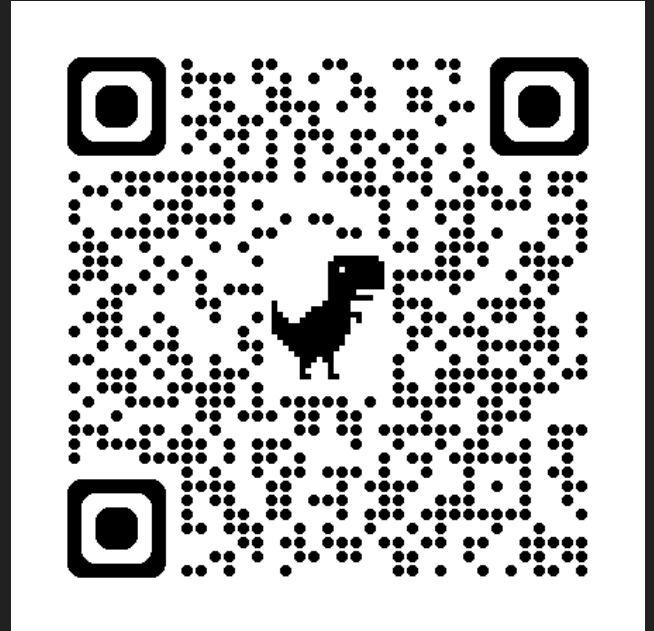
Read the question, choose the answer and submit.

The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

**If you are not in the classroom, a report will
be sent to DADAC!**



Divide-and-Conquer

- Some *algorithms* use a Divide-and-Conquer strategy (a.k.a. Divide-Conquer-and-Combine)
 - A problem instance is divided into two or more smaller problems
 - The smaller instances are solved recursively
 - The results are then combined to get the result for the original instance
- Two examples: Merge Sort and Quicksort

```
resultType f(inputType x) {  
  
    if (baseCase(x))  
        return handleBaseCase(x);  
  
    inputType x1, x2;  
    Divide(x,x1,x2);  
  
    resultType y1, y2;  
    y1 = f(x1); y2 = f(x2);  
  
    return Combine(y1,y2);  
}
```


Merge Sort

- Split into two (almost) equal halves, and recursively sort each half
- Merge the two sorted arrays into a single sorted array

53	46	94	43	17	12	60	98	86	50	36	26	57	80	77	18
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----



Recursively
sort

Recursively
sort



12	17	43	46	53	60	94	98
----	----	----	----	----	----	----	----

18	26	36	50	57	77	80	86
----	----	----	----	----	----	----	----

Merge

12	17	18	26	36	43	46	50	53	57	60	77	80	86	94	98
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Merge Sort

```
// merge X[left..mid] and X[mid+1..right] into Y[left..right]
void merge(const vector<int>& X, int left, int mid, int right, vector<int>& Y) {
    int L = left, R = mid+1;    // L,R: next indices of left/right halves
    for(int i=left; i <= right; ++i) {
        if(L <= mid && (R > right || X[L] <= X[R])) Y[i] = X[L++];    // copy from left
        else Y[i] = X[R++];    // copy from right
    }
}
```

12	17	43	46	53	60	94	98
----	----	----	----	----	----	----	----

left

mid

18	26	36	50	57	77	80	86
----	----	----	----	----	----	----	----

mid+1

right

Merge

12	17	18	26	36	43	46	50	53	57	60	77	80	86	94	98
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

left

i →

right

Merge Sort

```
void sort (const std::vector<int>& in, int left,  
          int right, std::vector<int>& out,  
          std::vector<int>& aux) {
```

```
    if (left == right)  
        out[left] = in[left];  
    if (left >= right)  
        return;
```

```
    int mid = (left+right)/2;
```

```
    sort(in, left, mid, aux, out); // outputs into aux  
    sort(in, mid+1, right, aux, out);
```

```
    merge(aux, left, mid, right, out); // aux is input
```

```
}
```

Output will be in `out[left, ..., right]`.
A temporary vector `aux` is passed (to
avoid allocating $\Theta(n)$ space in each
recursive call)

Total extra space is
 $O(n) + O(\log n) = O(n)$

```
resultType f(inputType x) {
```

```
    if (baseCase(x))  
        return handleBaseCase(x);
```

```
    inputType x1, x2;  
    Divide(x, x1, x2);
```

```
    resultType y1, y2;  
    y1 = f(x1); y2 = f(x2);
```

```
    return Combine(y1, y2);
```

```
}
```

Merge Sort

```
void sort (const std::vector<int>& in, int left,
          int right, std::vector<int>& out,
                std::vector<int>& aux) {

    if (left == right)
        out[left] = in[left];
    if (left >= right)
        return;

    int mid = (left+right)/2;

    sort(in, left, mid, aux, out);
    sort(in, mid+1, right, aux, out);

    merge(aux, left, mid, right, out);
}
```

- Time?
 - $T(n) = 2T(n/2) + O(n)$
 - $O(n \log n)$